

Guia de estudio - API de Matricula

Programacion III - entrega y defensa oral

Este documento resume que debes revisar, como ejecutar el proyecto y como explicar la arquitectura sin memorizar todo el codigo.

Checklist de entrega

Elemento	Estado
Repositorio GitHub	Publicado: Matricula-api-valeria
uml.png	Creado y subido
API modular	app/, routers/, services/ y main.py
Pruebas	12 passed
Archivos excluidos	venv, .venv, cache y base de datos generada

Comandos importantes

Desde la carpeta API_Sistema_Academico_FastAPI_SQLite:

```
python -m uvicorn main:app --reload  
python -m pytest -q
```

Abre el navegador en <http://127.0.0.1:8000/docs> para Swagger.

Arquitectura que debes explicar

Archivo	Responsabilidad
main.py	Punto de entrada corto: importa app desde app.main.
app/main.py	Crea FastAPI, inicializa SQLite y registra routers.
app/database.py	Conexiones, tablas, indices y transacciones SQLite.
app/schemas.py	Valida los datos de entrada y salida.
app/routers/	Define endpoints HTTP por modulo.
app/services/	Aplica reglas de negocio complejas.
frontend/	Login y dashboard servidos por la misma API.

Flujo de una matricula

1. El cliente envia JSON a /api/v1/matriculas. 2. El router recibe y valida el esquema. 3. matricula_service comprueba las reglas. 4. database abre una transaccion. 5. SQLite inserta o actualiza. 6. El router devuelve la respuesta JSON.

Reglas esenciales

- El estudiante debe estar activo.
- El periodo debe estar activo y la seccion abierta.
- No puede repetirse la misma matricula.
- No se puede superar el cupo ni 20 unidades valorativas.
- No se permiten choques de horario.
- Cancelar cambia el estado y libera el cupo sin borrar el historial.

Preguntas probables del ingeniero

Que hace APIRouter?

Agrupar endpoints relacionados para registrarlos luego con `app.include_router`.

Por que separar routers y services?

El router atiende HTTP; el service concentra las reglas de negocio y facilita probarlas.

Por que usar esquemas?

Validan tipos, identificadores y datos antes de ejecutar SQL.

Por que usar transacciones?

Confirman todos los cambios juntos o hacen rollback si ocurre un error.

Como se evita la inyeccion SQL?

Las consultas usan parametros con signos `?`, nunca concatenan datos del usuario.

Como funciona el login?

Busca el usuario, compara el hash de la clave y verifica que la cuenta este activa.

Como se calcula una nota?

Se guardan tres parciales, se calcula la nota final y con 65 o mas queda APROBADA.

Que muestra uml.png?

Usuario, Estudiante, Curso/Asignatura y Matricula, con sus atributos y relaciones.

Diferencia frente a la guia

La guia menciona una API inicial monolitica y nombres como `seguridad.py`, `esquemas.py`, `cursos.py`, `dashboard.py` y `paginas.py`. Este repositorio es una version mas completa y ya modularizada: usa `auth.py`, `schemas.py`, `asignaturas.py`, `reportes.py` y `frontend`. No renombres modulos sin el `main.py` original del ingeniero, porque podrias romper la API.

Repasa primero el flujo de una matricula y luego las reglas de negocio. Eso te permite explicar el proyecto aunque no recuerdes cada linea.